

1 Scope and Data Flow

This document contains only the dynamic-obstacle pipeline implemented in the current codebase:

LiDAR cluster $\rightarrow$ track association $\rightarrow$ obstacle EKF $\rightarrow$ TCPA/DCPA $\rightarrow$ virtual fields $\rightarrow$ APF.

The vessel six-state EKF, thruster dynamics, COLREG decision details, route controller, GUI, and unrelated source files are excluded.

2 Obstacle EKF

2.1 State and Constant-Velocity Model

Each obstacle track uses

$$\mathbf{x}_k = [p_N \ p_E \ v_N \ v_E]^\top,$$

where p_N, p_E are North-East position components and v_N, v_E are the corresponding velocities. The transition model is

$$\mathbf{x}_{k+1} = \mathbf{F}(\Delta t)\mathbf{x}_k, \quad \mathbf{F} = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Position is advanced by velocity multiplied by the time step, while velocity is copied forward.

2.2 Process Noise

The constant-velocity assumption is imperfect because a real obstacle may accelerate or turn. The process covariance models this uncertainty:

$$\mathbf{Q} = q \begin{bmatrix} \frac{\Delta t^4}{4} & 0 & \frac{\Delta t^3}{2} & 0 \\ 0 & \frac{\Delta t^4}{4} & 0 & \frac{\Delta t^3}{2} \\ \frac{\Delta t^3}{2} & 0 & \Delta t^2 & 0 \\ 0 & \frac{\Delta t^3}{2} & 0 & \Delta t^2 \end{bmatrix}, \quad q = \sigma_a^2.$$

Larger σ_a makes the filter less confident in constant velocity and more responsive to measurements.

```
def obstacle_ekf_process_noise(self, dt):
    dt = max(float(dt), 1e-3)
    q = self.obstacle_ekf_accel_std_m_s2 ** 2
    dt2 = dt * dt
    dt3 = dt2 * dt
    dt4 = dt2 * dt2

    return q * np.array(
        [
            [0.25 * dt4, 0.0, 0.5 * dt3, 0.0],
            [0.0, 0.25 * dt4, 0.0, 0.5 * dt3],
            [0.5 * dt3, 0.0, dt2, 0.0],
            [0.0, 0.5 * dt3, 0.0, dt2],
        ],
        dtype=float,
    )
```

```

def obstacle_ekf_predict(self, state, covariance, dt):
    dt = max(float(dt), 0.0)
    state = np.asarray(state, dtype=float).reshape(4)
    covariance = np.asarray(covariance, dtype=float).reshape(4, 4)

    F = np.array(
        [
            [1.0, 0.0, dt, 0.0],
            [0.0, 1.0, 0.0, dt],
            [0.0, 0.0, 1.0, 0.0],
            [0.0, 0.0, 0.0, 1.0],
        ],
        dtype=float,
    )

    predicted_state = F @ state
    predicted_covariance = F @ covariance @ F.T + self.obstacle_ekf_process_noise(
        dt)
    return predicted_state, predicted_covariance

```

2.3 Prediction and Position Update

The prediction is

$$\hat{\mathbf{x}}_k = \mathbf{F}\mathbf{x}_{k-1}^+, \text{quad} \hat{\mathbf{P}}_k = \mathbf{F}\mathbf{P}_{k-1}^+ \mathbf{F}^\top + \mathbf{Q}.$$

LiDAR measures position only:

$$\mathbf{z} = [p_N^{\text{meas}} \quad p_E^{\text{meas}}]^\top, \text{quad} \mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}.$$

The innovation, gain, corrected state, and Joseph covariance update are

$$\boldsymbol{\nu} = \mathbf{z} - \mathbf{H}\hat{\mathbf{x}}, \quad (1)$$

$$\mathbf{S} = \mathbf{H}\hat{\mathbf{P}}\mathbf{H}^\top + \mathbf{R}, \quad (2)$$

$$\mathbf{K} = \hat{\mathbf{P}}\mathbf{H}^\top \mathbf{S}^{-1}, \quad (3)$$

$$\mathbf{x}^+ = \hat{\mathbf{x}} + \mathbf{K}\boldsymbol{\nu}, \quad (4)$$

$$\mathbf{P}^+ = (\mathbf{I} - \mathbf{K}\mathbf{H})\hat{\mathbf{P}}(\mathbf{I} - \mathbf{K}\mathbf{H})^\top + \mathbf{K}\mathbf{R}\mathbf{K}^\top. \quad (5)$$

The covariance expresses uncertainty: the state is the best estimate, while the covariance determines how strongly the next measurement should correct it.

```

def obstacle_measurement_covariance(self, measurement_ne, reference_pose=None):
    measurement_ne = np.asarray(measurement_ne, dtype=float).reshape(2)
    if reference_pose is None:
        reference_pose = getattr(self, "lidar_reference_pose", None)
    if reference_pose is None:
        reference_pose = np.asarray(self.p_robot, dtype=float).reshape(3)
    else:
        reference_pose = np.asarray(reference_pose, dtype=float).reshape(3)

    base_variance = float(self.obstacle_ekf_measurement_std_m) ** 2
    yaw_variance = 0.0
    sigma = getattr(self, "Sigma", None)
    if sigma is not None:
        sigma = np.asarray(sigma, dtype=float)
        if sigma.ndim == 2 and sigma.shape[0] > G and sigma.shape[1] > G:

```

```

        yaw_variance = max(float(sigma[G, G]), 0.0)

        yaw_rate_rad_s = getattr(self, "sensed_imu_yaw_rate_rad_s", None)
        if yaw_rate_rad_s is None or not np.isfinite(yaw_rate_rad_s):
            velocity = np.asarray(getattr(self, "v_robot", np.zeros((3, 1))), dtype=
float).reshape(-1)
            yaw_rate_rad_s = velocity[2] if len(velocity) > 2 else 0.0

        timing_std_s = max(
            float(getattr(self, "lidar_time_sync_std_s", 0.02)),
            float(getattr(self, "lidar_scan_time_s", 0.0)) / np.sqrt(12.0),
        )
        yaw_variance += (float(yaw_rate_rad_s) * timing_std_s) ** 2

        relative_ne = measurement_ne - reference_pose[0:2]
        yaw_jacobian = np.array([-relative_ne[1], relative_ne[0]], dtype=float)
        return (
            base_variance * np.eye(2, dtype=float)
            + yaw_variance * np.outer(yaw_jacobian, yaw_jacobian)
        )

def obstacle_ekf_update(
    self,
    state,
    covariance,
    measurement_ne,
    measurement_covariance=None,
):
    state = np.asarray(state, dtype=float).reshape(4)
    covariance = np.asarray(covariance, dtype=float).reshape(4, 4)
    measurement_ne = np.asarray(measurement_ne, dtype=float).reshape(2)

    H = np.array(
        [
            [1.0, 0.0, 0.0, 0.0],
            [0.0, 1.0, 0.0, 0.0],
        ],
        dtype=float,
    )
    R_obs = (
        self.obstacle_measurement_covariance(measurement_ne)
        if measurement_covariance is None
        else np.asarray(measurement_covariance, dtype=float).reshape(2, 2)
    )
    innovation = measurement_ne - H @ state
    innovation_covariance = H @ covariance @ H.T + R_obs

    try:
        kalman_gain = covariance @ H.T @ np.linalg.inv(innovation_covariance)
    except np.linalg.LinAlgError:
        kalman_gain = covariance @ H.T @ np.linalg.pinv(innovation_covariance)

    corrected_state = state + kalman_gain @ innovation
    identity = np.eye(4, dtype=float)
    correction = identity - kalman_gain @ H
    corrected_covariance = correction @ covariance @ correction.T + kalman_gain @
R_obs @ kalman_gain.T
    return corrected_state, corrected_covariance

```

2.4 Track Stability and Future Position

A track is not accepted as a reliable moving obstacle after a single detection. The implementation checks sample count, hit count, time span, speed, and missed detections. Stable tracks are extrapolated as

$$\mathbf{p}(\tau) = \mathbf{p}_0 + \mathbf{v}_0\tau + \frac{1}{2}\mathbf{a}\tau^2.$$

When acceleration is unavailable or too uncertain, $\mathbf{a} = \mathbf{0}$ and the model becomes constant velocity.

```
def obstacle_track_motion_is_stable(self, track):
    sample_count = int(track.get("stats_sample_count", 0))
    hit_count = int(track.get("hit_count", 0))
    if (
        sample_count < self.obstacle_prediction_min_samples
        or hit_count < self.obstacle_prediction_min_hits
        or int(track.get("miss_count", 0)) > 0
    ):
        return False

    samples = track.get("motion_window", [])
    if len(samples) < 2:
        return False

    first_stamp = float(samples[0].get("stamp_s", 0.0))
    last_stamp = float(samples[-1].get("stamp_s", first_stamp))
    if last_stamp - first_stamp < self.obstacle_prediction_min_time_span_s:
        return False

    speed_m_s = float(track.get("speed_mean_m_s", 0.0))
    was_stable = bool(track.get("motion_stable", False))
    speed_threshold = (
        self.apf_dynamic_exit_speed_threshold_m_s
        if was_stable
        else self.apf_dynamic_speed_threshold_m_s
    )
    if not np.isfinite(speed_m_s) or speed_m_s < speed_threshold:
        return False

    return True

def obstacle_track_prediction_accel_ne(self, track):
    if (
        not bool(track.get("motion_stable", False))
        or int(track.get("stats_sample_count", 0)) < self.
obstacle_prediction_accel_min_samples
    ):
        return np.zeros(2, dtype=float)

    accel_ne = np.asarray(track.get("accel_ne", [0.0, 0.0]), dtype=float).reshape
(2)
    accel_var_ne = np.asarray(track.get("accel_var_ne", [np.inf, np.inf]), dtype=
float).reshape(2)
    if not np.isfinite(accel_ne).all() or not np.isfinite(accel_var_ne).all():
        return np.zeros(2, dtype=float)

    accel_std_m_s2 = float(np.sqrt(max(float(np.max(accel_var_ne)), 0.0)))
    if accel_std_m_s2 > self.obstacle_prediction_max_accel_std_m_s2:
        return np.zeros(2, dtype=float)
```

```

        return accel_ne

def obstacle_track_prediction_ne(self, track):
    if not self.obstacle_ekf_prediction_enabled:
        return np.empty((0, 2), dtype=float)

    state = np.asarray(track.get("state", [np.nan, np.nan, 0.0, 0.0]), dtype=float)
    .reshape(4)
    if not np.isfinite(state).all():
        return np.empty((0, 2), dtype=float)

    velocity_ne = state[2:4].copy()

    accel_ne = self.obstacle_track_prediction_accel_ne(track)

    step_s = max(float(self.obstacle_prediction_step_s), 1e-3)
    horizon_s = float(self.obstacle_prediction_horizon_s)
    if not np.isfinite(horizon_s) or horizon_s <= 0.0:
        return np.empty((0, 2), dtype=float)
    times = np.arange(0.0, horizon_s, step_s, dtype=float)
    times = np.r_[times, horizon_s]

    return np.column_stack(
        [
            state[0] + velocity_ne[0] * times + 0.5 * accel_ne[0] * times * times,
            state[1] + velocity_ne[1] * times + 0.5 * accel_ne[1] * times * times,
        ]
    )

```

```

def update_apf_obstacle_tracks(self, stamp_s):
    if not self.obstacle_ekf_prediction_enabled:
        self.apf_obstacle_tracks = []
        self.apf_virtual_obstacles = []
        tracked_fields = {
            "track_id",
            "velocity_ne",
            "velocity_mean_ne",
            "velocity_var_ne",
            "accel_ne",
            "speed_m_s",
            "speed_mean_m_s",
            "speed_var_m2_s2",
            "heading_rad",
            "heading_deg",
            "heading_mean_rad",
            "heading_var_rad2",
            "pc1_m",
            "pc2_m",
            "pc1_var_m2",
            "pc2_var_m2",
            "stats_sample_count",
            "motion_stable",
            "prediction_model",
            "predicted_trajectory_ne",
        }
        for obstacle in self.lidar_obstacles:
            for field in tracked_fields:
                obstacle.pop(field, None)
        return

```

```

        now = float(stamp_s if stamp_s is not None else time.time())
        confirmation_hits = max(int(getattr(self, "obstacle_track_confirmation_hits",
2)), 2)
        detections = []

        for obstacle_index, obstacle in enumerate(self.lidar_obstacles):
            centre_ne = np.asarray(obstacle.get("centre_ne", [np.nan, np.nan]), dtype=
float).reshape(2)
            if np.isfinite(centre_ne).all():
                pc1_m = float(obstacle.get("pc1_m", self.obstacle_min_pc1_m))
                pc2_m = float(obstacle.get("pc2_m", self.obstacle_min_pc2_m))
                length_axis_ne = np.asarray(
                    obstacle.get("length_axis_ne", [1.0, 0.0]),
                    dtype=float,
                ).reshape(2)
                measurement_covariance = np.asarray(
                    obstacle.get(
                        "measurement_covariance",
                        self.obstacle_measurement_covariance(centre_ne),
                    ),
                    dtype=float,
                ).reshape(2, 2)
                detections.append(
                    (
                        obstacle_index,
                        centre_ne,
                        pc1_m,
                        pc2_m,
                        length_axis_ne,
                        measurement_covariance,
                    )
                )

            if not detections:
                self.apf_obstacle_track_candidates = [
                    candidate
                    for candidate in self.apf_obstacle_track_candidates
                    if now - float(candidate.get("last_seen_s", candidate.get("stamp_s",
now))) <= self.apf_track_timeout_s
                ]
                for track in self.apf_obstacle_tracks:
                    self.predict_obstacle_track_to_time(track, now)
                    track["miss_count"] = int(track.get("miss_count", 0)) + 1

                self.prune_obstacle_tracks(now)
                return

            detection_positions = np.asarray(
                [detection for _, detection, _, _, _ in detections],
                dtype=float,
            )
            predicted_tracks = []
            candidates = []
            matched_candidates = set()
            promoted_candidates = set()

            for track_index, track in enumerate(self.apf_obstacle_tracks):
                dt = max(now - float(track.get("stamp_s", now)), 0.0)

```

```

        predicted_state, predicted_covariance = self.obstacle_ekf_predict(
            track.get("state", np.r_[track.get("pos_ne", [np.nan, np.nan]), track.
get("vel_ne", [0.0, 0.0])]),
            track.get("covariance", np.eye(4, dtype=float)),
            dt,
        )
        predicted_tracks.append((predicted_state, predicted_covariance))
        predicted_pos = predicted_state[0:2]

        for detection_index, detection in enumerate(detection_positions):
            distance = float(np.linalg.norm(detection - predicted_pos))
            candidates.append((distance, track_index, detection_index))

candidates.sort(key=lambda item: item[0])
assigned_tracks = set()
assigned_detections = set()
detection_track = {}

for distance, track_index, detection_index in candidates:
    if distance > self.apf_track_association_m:
        break
    if track_index in assigned_tracks or detection_index in assigned_detections
:
        continue

    track = self.apf_obstacle_tracks[track_index]
    detection = detection_positions[detection_index]
    predicted_state, predicted_covariance = predicted_tracks[track_index]
    corrected_state, corrected_covariance = self.obstacle_ekf_update(
        predicted_state,
        predicted_covariance,
        detection,
        detections[detection_index][5],
    )

    track["state"] = corrected_state
    track["covariance"] = corrected_covariance
    track["stamp_s"] = now
    track["last_seen_s"] = now
    track["hit_count"] = int(track.get("hit_count", 0)) + 1
    track["miss_count"] = 0
    self.sync_obstacle_track_fields(track)
    self.append_obstacle_track_history(
        track,
        pc1_m=detections[detection_index][2],
        pc2_m=detections[detection_index][3],
        length_axis_ne=detections[detection_index][4],
        detection_ne=detection,
        stamp_s=now,
    )
    assigned_tracks.add(track_index)
    assigned_detections.add(detection_index)
    detection_track[detection_index] = track

for track_index, track in enumerate(self.apf_obstacle_tracks):
    if track_index not in assigned_tracks:
        predicted_state, predicted_covariance = predicted_tracks[track_index]
        track["state"] = predicted_state
        track["covariance"] = predicted_covariance

```

```

        track["stamp_s"] = now
        track["miss_count"] = int(track.get("miss_count", 0)) + 1
        self.sync_obstacle_track_fields(track)

    for detection_index, detection in enumerate(detections):
        if detection_index in assigned_detections:
            continue

        _, detection_ne, pc1_m, pc2_m, length_axis_ne, _ = detection
        best_candidate_index = None
        best_candidate_distance = np.inf
        for candidate_index, candidate in enumerate(self.
apf_obstacle_track_candidates):
            if candidate_index in matched_candidates:
                continue
            candidate_pos = np.asarray(candidate.get("centre_ne", [np.nan, np.nan])
, dtype=float).reshape(2)
            if not np.isfinite(candidate_pos).all():
                continue
            distance = float(np.linalg.norm(detection_ne - candidate_pos))
            if distance < best_candidate_distance:
                best_candidate_distance = distance
                best_candidate_index = candidate_index

        if best_candidate_index is not None and best_candidate_distance <= self.
apf_track_association_m:
            candidate = self.apf_obstacle_track_candidates[best_candidate_index]
            candidate["centre_ne"] = detection_ne.copy()
            candidate["last_seen_s"] = now
            candidate["hit_count"] = int(candidate.get("hit_count", 0)) + 1
            candidate["pc1_m"] = pc1_m
            candidate["pc2_m"] = pc2_m
            candidate["length_axis_ne"] = length_axis_ne
            matched_candidates.add(best_candidate_index)
            if int(candidate["hit_count"]) >= confirmation_hits:
                track = self.make_obstacle_track(
                    detection_ne,
                    now,
                    pc1_m=pc1_m,
                    pc2_m=pc2_m,
                    length_axis_ne=length_axis_ne,
                )
                self.apf_obstacle_tracks.append(track)
                assigned_detections.add(detection_index)
                detection_track[detection_index] = track
                promoted_candidates.add(best_candidate_index)
            continue

        self.apf_obstacle_track_candidates.append(
            self.make_obstacle_track_candidate(
                detection_ne,
                now,
                pc1_m=pc1_m,
                pc2_m=pc2_m,
                length_axis_ne=length_axis_ne,
            )
        )

    for detection_index, track in detection_track.items():

```

```

        obstacle_index, _, _, _, _ = detections[detection_index]
        self.annotate_lidar_obstacle_with_track(self.lidar_obstacles[obstacle_index], track)

        self.apf_obstacle_track_candidates = [
            candidate
            for candidate_index, candidate in enumerate(self.apf_obstacle_track_candidates)
            if candidate_index not in promoted_candidates
            if now - float(candidate.get("last_seen_s", candidate.get("stamp_s", now)))
            <= self.apf_track_timeout_s
        ]
        self.prune_obstacle_tracks(now)

```

3 TCPA and DCPA

Let own-ship position and velocity be $(\mathbf{p}_o, \mathbf{v}_o)$, and obstacle position and velocity be $(\mathbf{p}_t, \mathbf{v}_t)$. The implementation assumes that both vehicles keep their current velocities over the short prediction interval.

The relative position and velocity are

$$\mathbf{r} = \mathbf{p}_t - \mathbf{p}_o, \quad \mathbf{v}_r = \mathbf{v}_t - \mathbf{v}_o.$$

The squared separation at time t is

$$d^2(t) = \|\mathbf{r} + \mathbf{v}_r t\|^2$$

and therefore

$$d^2(t) = \mathbf{r}^\top \mathbf{r} + 2t \mathbf{r}^\top \mathbf{v}_r + t^2 \mathbf{v}_r^\top \mathbf{v}_r.$$

Minimising this quadratic function gives the unconstrained closest-approach time

$$t^* = -\frac{\mathbf{r}^\top \mathbf{v}_r}{\|\mathbf{v}_r\|^2}.$$

3.1 How the Sign Determines Closing or Separating

The key quantity is $\mathbf{r}^\top \mathbf{v}_r$:

- If it is negative, the vehicles are closing and $t^* > 0$; the closest point is in the future.
- If it is positive, the vehicles are separating and $t^* < 0$; the mathematical closest point is in the past, so the current instant is used.
- If it is zero, relative motion is perpendicular to the current separation and the current instant is the closest point under the straight-line model.
- If $\|\mathbf{v}_r\|^2$ is nearly zero, there is no reliable relative-motion direction; the implementation avoids division by a nearly zero value.

The implementation clips the non-negative time to the configured horizon:

$$t_{\text{CPA}} = \min(\max(t^*, 0), T_{\text{max}}).$$

Thus, if the unrestricted closest point lies beyond the horizon, the reported distance is the distance at the horizon rather than an unrestricted long-term DCPA.

The CPA positions and distance are

$$\mathbf{p}_{o,CPA} = \mathbf{p}_o + \mathbf{v}_o t_{CPA}, \quad \mathbf{p}_{t,CPA} = \mathbf{p}_t + \mathbf{v}_t t_{CPA},$$

$$d_{CPA} = \|\mathbf{p}_{t,CPA} - \mathbf{p}_{o,CPA}\|.$$

TCPA answers “when will the separation be smallest?”; DCPA answers “how large will that separation be at the reported time?”.

3.2 Numerical Examples

For $\mathbf{r} = [10, -5]^\top$ and $\mathbf{v}_r = [-1, 1]^\top$, the dot product is -15 and the relative-speed square is 2 . Therefore $t^* = 7.5$ s, so a 10 s horizon returns $t_{CPA} = 7.5$ s.

For $\mathbf{r} = [10, 0]^\top$ and $\mathbf{v}_r = [1, 0]^\top$, $t^* = -10$ s. The code clips it to zero and reports the current separation as DCPA.

```

        break

    result = labels.copy()
    for merged_label, group in enumerate(groups):
        result[np.isin(labels, group)] = merged_label
    return result

def straight_line_cpa(own_position, own_velocity, obstacle_position, obstacle_velocity,
    horizon_s=None):
    """Continuous-time CPA for two constant-velocity trajectories."""
    own_position = np.asarray(own_position, dtype=float).reshape(2)
    own_velocity = np.asarray(own_velocity, dtype=float).reshape(2)
    obstacle_position = np.asarray(obstacle_position, dtype=float).reshape(2)
    obstacle_velocity = np.asarray(obstacle_velocity, dtype=float).reshape(2)
    relative_position = obstacle_position - own_position
    relative_velocity = obstacle_velocity - own_velocity
    relative_speed_sq = float(np.dot(relative_velocity, relative_velocity))
    tcpa_s = 0.0 if relative_speed_sq < 1e-12 else max(
        -float(np.dot(relative_position, relative_velocity)) / relative_speed_sq,
        0.0,
    )
    if horizon_s is not None:
        tcpa_s = min(tcpa_s, max(float(horizon_s), 0.0))
    own_cpa = own_position + own_velocity * tcpa_s
    obstacle_cpa = obstacle_position + obstacle_velocity * tcpa_s
    return tcpa_s, float(np.linalg.norm(obstacle_cpa - own_cpa)), own_cpa, obstacle_cpa

def apf_cpa_metrics(self, obs_pos_body, obs_vel_body, own_vel_body):
    if not self.obstacle_ekf_prediction_enabled:
        return np.nan, np.nan
    tcpa_s, dcpa_m, _, _ = straight_line_cpa(
        [0.0, 0.0], own_vel_body, obs_pos_body, obs_vel_body
    )
    return tcpa_s, dcpa_m

```

4 TCPA-Gated Virtual APF Fields

Only a stable, sufficiently fast track satisfying

$$0 < t_{\text{CPA}} \leq T_{\text{horizon}}$$

is converted into a virtual field. The final field centre is placed at

$$\mathbf{p}_{\text{virtual}} = \mathbf{p}_t + 2t_{\text{CPA}}\mathbf{v}_t.$$

The factor two places the predicted field ahead of the closest-approach point and gives the planner time to react; it is an implementation rule, not a physical law.

4.1 Own-Ship Equivalent Radius

The obstacle is represented by an ellipse, while planning uses the own-ship reference point. The field therefore expands the obstacle semi-axes:

$$a = \frac{pc1}{2} + r_{\text{own}}, \quad b = \frac{pc2}{2} + r_{\text{own}}.$$

This approximates the own ship as a circle and inflates the obstacle footprint. Without this margin, the own-ship reference point could clear an obstacle while the physical hull still intersects it.

4.2 Bridge Fields

Intermediate ellipses are inserted between the current predicted position and the final $2t_{\text{CPA}}$ position. Their spacing is based on the combined support radius, producing a continuous predicted exclusion corridor instead of isolated future points.

```
def obstacle_track_state_at(self, track, dt_s):
    """Predict the 4-D [north, east, v_north, v_east] EKF state."""
    state = np.asarray(track.get("state", [np.nan] * 4), dtype=float).reshape(4)
    position_ne = state[:2]
    velocity_ne = state[2:4]
    if not np.isfinite(position_ne).all() or not np.isfinite(velocity_ne).all():
        return None, None
    return position_ne + velocity_ne * max(float(dt_s), 0.0), velocity_ne.copy()

def update_apf_virtual_obstacles(self):
    """Build the 2×TCPA field and overlapping bridge from each live EKF track."""
    self.apf_virtual_obstacles = []
    if not self.obstacle_ekf_prediction_enabled:
        return self.apf_virtual_obstacles

    own_start_ne = self._current_ne()
    measured_velocity_ne = np.asarray(self.v_robot[0:2], dtype=float).reshape(2)
    own_speed_m_s = float(np.linalg.norm(measured_velocity_ne))
    if not np.isfinite(own_speed_m_s) or own_speed_m_s < 1e-6:
        own_speed_m_s = float(self.route_tracking_speed_m_s)
    own_velocity_ne = self.body_vector_to_earth(np.array([own_speed_m_s, 0.0]))
    horizon_s = float(self.apf_collision_horizon_s)

    for track in self.apf_obstacle_tracks:
        if not self.obstacle_track_motion_is_stable(track):
            continue
```

```

start_ne, velocity_ne = self.obstacle_track_state_at(track, 0.0)
if start_ne is None or velocity_ne is None:
    continue
start_ne = np.asarray(start_ne, dtype=float).reshape(2)
velocity_ne = np.asarray(velocity_ne, dtype=float).reshape(2)
speed_m_s = float(np.linalg.norm(velocity_ne))
if not np.isfinite(start_ne).all() or not np.isfinite(velocity_ne).all() or
speed_m_s < self.apf_dynamic_speed_threshold_m_s:
    continue
tcpa_s, dcpa_m, own_cpa_ne, obstacle_cpa_ne = straight_line_cpa(
    own_start_ne, own_velocity_ne, start_ne, velocity_ne
)
if not 0.0 < tcpa_s <= horizon_s:
    continue

pc1_m, pc2_m = self.obstacle_pc_dimensions(track)
axis_ne = velocity_ne / speed_m_s
final_ne = start_ne + velocity_ne * (2.0 * tcpa_s)
field_half_along_m = self.apf_predicted_field_size_scale * (
    0.5 * pc1_m + self.apf_own_equivalent_radius_m
)
field_half_lateral_m = self.apf_predicted_field_size_scale * (
    0.5 * pc2_m + self.apf_own_equivalent_radius_m
)
base_radius_m = min(pc1_m, pc2_m) * 0.5 + self.apf_own_equivalent_radius_m
max_gap_m = max(
    (self.apf_current_field_size_scale + self.
apf_predicted_field_size_scale) * base_radius_m,
    1e-6,
)
bridge_count = max(int(np.ceil(np.linalg.norm(final_ne - start_ne) /
max_gap_m)) - 1, 0)

def field(position_ne, bridge=False):
    return {
        "label": -1000 - int(track.get("id", 0)), "virtual": True,
        "bridge": bridge, "track_id": int(track.get("id", 0)),
        "centre_ne": np.asarray(position_ne, dtype=float).tolist(),
        "centre_body": self.earth_point_to_body(position_ne).tolist(),
        "pc1_m": pc1_m, "pc2_m": pc2_m,
        "field_half_along_m": field_half_along_m,
        "field_half_lateral_m": field_half_lateral_m,
        "length_axis_ne": axis_ne.tolist(), "apf_ellipse_axis_ne": axis_ne.
tolist(),
        "apf_ellipse_pc1_m": pc1_m, "apf_ellipse_pc2_m": pc2_m,
        "heading_rad": float(np.arctan2(velocity_ne[1], velocity_ne[0])),
        "velocity_ne": velocity_ne.tolist(), "tcpa_s": tcpa_s, "dcpa_m":
dcpa_m,
        "virtual_time_s": 2.0 * tcpa_s,
        "collision_position_ne": own_cpa_ne.tolist(),
        "obstacle_dcpa_position_ne": obstacle_cpa_ne.tolist(),
        "obstacle_min_separation_point_ne": obstacle_cpa_ne.tolist(),
        "own_prediction_velocity_ne": own_velocity_ne.tolist(),
        "own_prediction_heading_rad": float(self.Yaw),
        "predicted_risk_active": True,
    }

```

5 APF Generation

5.1 Ellipse Geometry and Repulsion

For obstacle centre $\mathbf{c}$, length axis $\hat{\mathbf{l}}$, width axis $\hat{\mathbf{w}}$, and grid point $\mathbf{p}$, define

$$\ell = (\mathbf{p} - \mathbf{c}) \cdot \hat{\mathbf{l}}, \quad w = (\mathbf{p} - \mathbf{c}) \cdot \hat{\mathbf{w}},$$

$$\rho = \sqrt{\left(\frac{\ell}{a}\right)^2 + \left(\frac{w}{b}\right)^2}.$$

The real-time field is active inside its outer scale s . Between $\rho = 1$ and $\rho = s$, the cubic fade is

$$u = \text{clip}\left(\frac{\rho - 1}{s - 1}, 0, 1\right), \quad qquad f_{\text{fade}} = 1 - (3u^2 - 2u^3),$$

and

$$\mathbf{F}_{\text{rep}} = k_{\text{rep}} f_{\text{fade}} \hat{\mathbf{d}},$$

where $\hat{\mathbf{d}}$ points away from the obstacle.

```

    return _OvertakingController.apf_pass_astern_side_from_velocity(self,
obs_vel_body)

# The APF has exactly two repulsive fields: the measured LiDAR ellipse and
# the EKF/CPA-predicted ellipse. COLREG still selects the controller, but
# never injects an additional force into either field.
def _dynamic_ellipse_repulsion(self, obstacle, outer_scale):
    centre_body = self._obstacle_body_position(obstacle)
    if not np.isfinite(centre_body).all():
        return np.zeros(2, dtype=float), False
    axis_body = self.earth_vector_to_body(
        self.obstacle_length_axis_ne(obstacle)
    )
    pc1_m, pc2_m = self.obstacle_pc_dimensions(obstacle)
    # Measured, bridge, and final virtual fields are all evaluated at
    # their actual map positions. DCPA is telemetry only, not a second
    # force location.
    evaluation_offset_body = -centre_body
    level, away = ellipse_level_and_away(
        evaluation_offset_body,
        axis_body,
        0.5 * pc1_m + self.apf_own_equivalent_radius_m,
        0.5 * pc2_m + self.apf_own_equivalent_radius_m,
    )
    if level >= float(outer_scale):
        return np.zeros(2, dtype=float), False
    # Smoothly fades from the hull boundary (level=1) to zero at the
    # requested size multiple; inside the measured ellipse remains unsafe.
    size_gain = 1.0 if bool(obstacle.get("raw_lidar_point", False)) else np.sqrt(
pc1_m * pc2_m)
    gain = self.apf_repulsive_gain * np.clip(size_gain, 0.5, 2.0)
    return smooth_ellipse_repulsion(

```

```

def smooth_ellipse_repulsion(level, away, gain, outer_scale):
    """Maximum at/inside the hull, C1-smooth to zero at the size multiple."""
    level = float(level)
    outer_scale = float(outer_scale)

```

```

if not np.isfinite(level) or outer_scale <= 1.0 or level >= outer_scale:
    return np.zeros(2, dtype=float), False
u = np.clip((level - 1.0) / (outer_scale - 1.0), 0.0, 1.0)
fade = 1.0 - (3.0 * u * u - 2.0 * u * u * u)
return float(gain) * fade * np.asarray(away, dtype=float).reshape(2), True

```

```

        away,
        gain,
        outer_scale,
    )

def apf_repulsion_for_obstacle(self, obstacle, target_body, own_vel_body):
    if getattr(self, "apf_selected_controller", "default_apf") == "overtaking":
        return _overtaking.LaptopController.apf_repulsion_for_obstacle(
            self, obstacle, target_body, own_vel_body
        )
    del target_body, own_vel_body
    force, active = self._dynamic_ellipse_repulsion(
        obstacle,
        float(getattr(
            self,
            "apf_predicted_field_size_scale"
            if bool(obstacle.get("virtual", False))
            else "apf_current_field_size_scale",
            4.0 if bool(obstacle.get("virtual", False)) else 2.5,
        )),
    )
    if active:
        self.apf_last_dynamic_field_s = (
            float(self.timefromstart) if self.timefromstart is not None else 0.0
        )
        obs_pos_body = self._obstacle_body_position(obstacle)
        obs_vel_body = self._obstacle_body_velocity(obstacle)
        own_vel_body = self.current_velocity_body()
        encounter, requested_side, action = self.apf_classify_encounter(
            obs_pos_body,
            obs_vel_body,
            own_vel_body,
        )
        if bool(getattr(self, "apf_head_on_latched", False)):
            encounter = "head_on"
            requested_side = -1.0
            action = "COLREG Rule 14: alter to starboard (latched)"
        elif bool(getattr(self, "apf_overtaking_latched", False)):
            encounter = "overtaking"
            requested_side = -1.0
            action = "COLREG Rule 13: overtake on starboard side (EKF-latched)"
        elif encounter in self._CROSSING_RULES:
            stern_side = self.apf_pass_astern_side_from_velocity(obs_vel_body)
            if stern_side != 0.0:
                requested_side = stern_side
                action = "Crossing: pass astern of obstacle ship"
        tcpa_s, dcpa_m = self.apf_cpa_metrics(
            obs_pos_body,
            obs_vel_body,
            own_vel_body,
        )
        if bool(getattr(self, "apf_overtaking_latched", False)):
            # Rule 13 must not inherit a port-side lock established by a

```

```

        # preceding crossing classification.
        self.apf_side_lock_sign = -1.0
        self.apf_side_lock_until_s = (
            float(self.timefromstart) if self.timefromstart is not None else
0.0
        ) + self.apf_side_lock_s
        self.apf_side_lock_active = True
        locked_side = -1.0
    else:
        locked_side = self.apf_lock_side(requested_side, 1.0)
        self.apf_encounter_mode = encounter

```

5.2 Attraction, Current Fields, and Virtual Fields

The snapshot APF is the sum of goal attraction, current-obstacle fields, and risk-active virtual fields:

$$U(\mathbf{p}) = U_{\text{goal}}(\mathbf{p}) + \sum_i U_{\text{current},i}(\mathbf{p}) + \sum_j U_{\text{virtual},j}(\mathbf{p}).$$

The goal potential is

$$U_{\text{goal}}(\mathbf{p}) = \frac{1}{2}k_g\|\mathbf{p} - \mathbf{p}_{\text{goal}}\|^2.$$

The real-time controller uses vector repulsion; the snapshot renderer uses scalar grid potentials. Both use the same obstacle centres, dimensions, axes, and virtual-field records.

```

def attractive_potential(north_grid, east_grid, target_ne, k_goal):
    target_ne = point(target_ne)
    if target_ne is None:
        return np.zeros_like(north_grid)
    return 0.5 * float(k_goal) * (
        (north_grid - target_ne[0]) ** 2
        + (east_grid - target_ne[1]) ** 2
    )

def classic_repulsive_potential(
    distance_grid,
    influence_distance_m,
    repulsive_gain,
):
    influence_distance_m = positive(influence_distance_m, 3.0)
    distance_grid = np.asarray(distance_grid, dtype=float)
    safe_distance = np.maximum(distance_grid, 1e-3)
    potential = np.zeros_like(safe_distance)
    active = safe_distance <= influence_distance_m
    potential[active] = 0.5 * float(repulsive_gain) * (
        1.0 / safe_distance[active] - 1.0 / influence_distance_m
    ) ** 2
    return potential

def ellipse_potential(
    north_grid,
    east_grid,
    obstacle,
    settings,
    k_obstacle=DEFAULT_K_OBSTACLE,

```

```

):
    centre = point(obstacle.get("centre_ne"))
    if centre is None:
        return np.zeros_like(north_grid)

    delta_n = north_grid - centre[0]
    delta_e = east_grid - centre[1]
    if settings.get("cluster_range_enabled", True) is False:
        return classic_repulsive_potential(
            np.hypot(delta_n, delta_e),
            settings.get("classic_influence_distance_m"),
            k_obstacle,
        )

    pc1_m, pc2_m = obstacle_dimensions(obstacle, settings)
    field_half_along_m = parse_float(obstacle.get("field_half_along_m"))
    field_half_lateral_m = parse_float(obstacle.get("field_half_lateral_m"))
    if np.isfinite(field_half_along_m) and np.isfinite(field_half_lateral_m):
        semi_length_m = max(field_half_along_m, 1e-6)
        semi_width_m = max(field_half_lateral_m, 1e-6)
    else:
        scale = positive(
            settings.get("avoidance_pc_scale"),
            settings.get("cluster_influence_scale", 6.0),
        )
        own_radius_m = positive(settings.get("own_equivalent_radius_m"), 0.0)
        semi_length_m = max(0.5 * scale * pc1_m + own_radius_m, 1e-6)
        semi_width_m = max(0.5 * scale * pc2_m + own_radius_m, 1e-6)
    axis = normalized_axis(obstacle.get("length_axis_ne"))
    width_axis = np.array([-axis[1], axis[0]], dtype=float)

    along = delta_n * axis[0] + delta_e * axis[1]
    across = delta_n * width_axis[0] + delta_e * width_axis[1]
    # Keep the plotted repulsive peak at centre_ne while preserving the
    # U=k/e domain boundary at the scaled pc1/pc2 ellipse.
    exponent = -(
        (along / semi_length_m) ** 2
        + (across / semi_width_m) ** 2
    )
    return float(k_obstacle) * np.exp(exponent)

```

```

def potential_components(
    north_grid,
    east_grid,
    payload,
    default_target_ne,
    k_goal,
    k_obstacle,
):
    settings = payload.get("apf_settings", {})
    settings = settings if isinstance(settings, dict) else {}
    apf = payload.get("apf", {})
    apf = apf if isinstance(apf, dict) else {}
    target_ne = point(apf.get("goal_ne"))
    if target_ne is None:
        target_ne = point(apf.get("path_end_ne"))
    if target_ne is None:
        target_ne = point(apf.get("target_ne"))
    if target_ne is None:

```

```

        target_ne = point(default_target_ne)
    attractive = attractive_potential(
        north_grid,
        east_grid,
        target_ne,
        k_goal,
    )
    current = np.zeros_like(north_grid)
    virtual = np.zeros_like(north_grid)
    tracks = [item for item in payload.get("tracks", []) if isinstance(item, dict)]
    virtuals = [
        item for item in payload.get("virtual_obstacles", [])
        if isinstance(item, dict) and bool(item.get("predicted_risk_active", False))
    ]
    for track in tracks:
        position_ne = point(track.get("position_ne"))
        if position_ne is None:
            continue
        current_obstacle = dict(
            track,
            centre_ne=position_ne,
            length_axis_ne=track_length_axis(track, payload),
        )
        current += ellipse_potential(north_grid, east_grid, current_obstacle, settings,
            k_obstacle)
    for field in virtuals:
        centre_ne = point(field.get("centre_ne"))
        if centre_ne is None:
            continue
        virtual += ellipse_potential(
            north_grid,
            east_grid,
            dict(field, centre_ne=centre_ne, length_axis_ne=field.get("
apf_ellipse_axis_ne")),
            settings,
            k_obstacle,
        )
    return attractive, current, virtual, target_ne

```

6 Minimal Code Map and Verification

Stage	Current implementation
EKF prediction	obstacle_ekf_predict
Process noise	obstacle_ekf_process_noise
Measurement update	obstacle_ekf_update
Stability and extrapolation	obstacle_track_motion_is_stable, obstacle_track_state_at
Track association	update_apf_obstacle_tracks
TCPA/DCPA	straight_line_cpa, apf_cpa_metrics
Virtual fields	update_apf_virtual_obstacles
Real-time APF	apf_repulsion_for_obstacle
Snapshot APF	potential_components

The focused regression checks are in `src/test_colreg_geometry.py`:

```
cd src
python -m pytest test_colreg_geometry.py -q
```